

**AGORAHUB: EMPOWERING COMMUNITIES
THROUGH COLLABORATIVE GOVERNANCE AND
INTELLIGENT DECISION-MAKING**

A MINI PROJECT REPORT

Submitted by

BALAGANESH K	2303717610421072
NANDHAKUMAR D	2303717610421102
SHAKE ASHIYA M	2303717610422117
SIBIRAJAN A	2303717610421121

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



COIMBATORE INSTITUTE OF TECHNOLOGY

(Government Aided Autonomous Institution Affiliated to Anna University)

COIMBATORE-641014

ANNA UNIVERSITY: CHENNAI 600 025

May 2026

COIMBATORE INSTITUTE OF TECHNOLOGY

(Government Aided Autonomous Institution Affiliated to Anna University)

COIMBATORE-641014

BONAFIDE CERTIFICATE

Certified that this mini project “**AGORAHUB: EMPOWERING COMMUNITIES THROUGH COLLABORATIVE GOVERNANCE AND INTELLIGENT DECISION-MAKING**” is the bonafide work of **Balaganesh K (2303717610421072), Nandhakumar D (2303717610421102), Shake ashiya M (2303717610422117), Sibirajan A (2303717610421121)** under my supervision during the academic year 2025-2026.

**Ms.V. PRIYANKA M.E.,
SUPERVISOR,**

Department of CSE,
Coimbatore Institute of Technology,
Coimbatore - 641014.

**Dr.A.KUNTHAVAI B.E., M.S., Ph.D.,
PROFESSOR AND HEAD,**

Department of CSE,
Coimbatore Institute of Technology,
Coimbatore - 641014.

Certified that the candidates were examined by us in the project work
viva-voce examination held on

Internal Examiner

External Examiner

Place:

Date:

ACKNOWLEDGEMENT

We express our sincere thanks to the CIT Management and to our Principal, **Dr. A. Rajeswari**, for providing us a great opportunity to carry out our work. The following words are a small express of our gratitude to them. This work is the outcome of their inspiration and the product of plethora of their knowledge and rich experience.

We record the deep sense of gratitude to **Dr. Kunthavai A, B.E., M.S., Ph.D.**, Head of the Department of Computer Science and Engineering, for her encouragement during this project.

We equally tender our sincere thankfulness to our project guide **Ms.V.Priyanka M.E.**, Assistant Professor, Department of Computer Science and Engineering, for her valuable suggestions and guidance during this project.

During the entire period of the project, all faculty members of the Department of Computer Science and Engineering have offered ungrudging help. It is also a great pleasure to acknowledge the unfailing help we have received from our friends.

It is a matter of great pleasure to thank our parents and family members for their constant support and cooperation in the pursuit of this endeavor.

Above all, I thank the Almighty for His blessings, which made this work possible.

ABSTRACT

AgoraHub is a unified digital platform developed to improve and simplify community governance in environments such as hostels, apartments, and clubs. It replaces traditional communication methods like notice boards and paper-based systems with a centralized mobile-based solution for managing announcements, complaints, petitions, and member activities efficiently. The platform ensures transparency, security, and active participation through features such as user registration, role-based access control, and real-time updates. AgoraHub also integrates Large Language Models (LLM) with Retrieval-Augmented Generation (RAG) to provide intelligent assistance in chat, complaints, and petition modules using community-specific data. Designed with a scalable and modular architecture, the system supports future enhancements like voting, analytics, and multilingual features while maintaining data privacy and accessibility. Overall, AgoraHub offers a modern, transparent, and organized approach to community management. The platform enhances communication between administrators and members by reducing delays and improving information accessibility. Its user-friendly interface encourages greater community participation and efficient issue resolution.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ACKNOWLEDGEMENT	i
	ABSTRACT	ii
	LIST OF TABLES	v
	LIST OF FIGURES	vi
	LIST OF ABBREVIATIONS	vii
1	INTRODUCTION	1
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	1
	1.3 OBJECTIVES OF THE PROPOSED SYSTEM	2
	1.4 APPLICATIONS OF THE PROJECT	3
2	LITERATURE SURVEY	4
	2.1 INTRODUCTION	4
	2.2 LITERATURE REVIEW OF RECENT WORKS	4
	2.3 SUMMARY OF LITERATURE SURVEY	9
	2.4 INFERENCES FROM LITERATURE REVIEW	10
	2.5 FINAL OBJECTIVES OF THE PROPOSED	11

	SYSTEM	
3	SYSTEM ARCHITECTURE	12
	3.1 ARCHITECTURE OVERVIEW	12
	3.2 ARCHITECTURE DIAGRAM	13
	3.3 ARCHITECTURE DESCRIPTION	14
	3.4 DATA FLOW DIAGRAM	26
	3.5 USECASE DIAGRAM	27
4	SYSTEM REQUIREMENTS	28
	4.1 HARDWARE REQUIREMENTS	28
	4.2 SOFTWARE REQUIREMENTS	28
5	SYSTEM DESIGN AND IMPLEMENTATION	29
	5.1 OVERVIEW OF THE SYSTEM DESIGN	29
	5.2 MODULES	29
6	RESULTS AND DISCUSSIONS	32
7	CONCLUSION AND FUTURE WORK	45
	REFERENCES	47

LIST OF TABLES

FIGURE NO	TITLE	PAGE NO
2.3	LITERATURE SURVEY	09

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
3.2	ARCHITECTURE DIAGRAM	13
3.4	DATA FLOW DIAGRAM	26
3.5	USECASE DIAGRAM	27
6.1	HOME PAGE	32
6.2	MANAGE MEMBERS	33
6.3	VOTING	34
6.4	COMPLAINT AI ASSISTANT	35
6.5	EVENTS	36
6.6	COMMUNITIES	37
6.7	CREATE COMMUNITY	38
6.8	COMMUNITY APP	39
6.9	VOTING ANALYSIS	40
6.10	VIEW COMPLAINTS	41
6.11	AI CHAT	42
6.12	CREATE VOTING - BASIC	43
6.13	CREATE VOTING - ADVANCED	44

LIST OF ABBREVIATIONS

API	- Application Programming Interface
IDE	- Integrated Development Environment
JSON	- JavaScript Object Notation
URL	- Uniform Resource Locator
ID	- Identifier
HTTP	- Hypertext Transfer Protocol
IP	- Internet Protocol
RBAC	- Role-Based Access Control (from previous context)
JS	- JavaScript (programming language)
DB	- Database (from db_mini)
JWT	- JSON Web Token
Req	- Request
Res	- Response
bcrypt	- Password hashing library (Blowfish Crypt)
Pg	- PostgreSQL (Node.js client for PostgreSQL)

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Community management plays an important role in organizing activities, enabling communication, and maintaining coordination within groups such as hostels, apartments, clubs, and organizations. Traditionally, these communities rely on methods like notice boards, physical meetings, and paper-based systems, which are often slow, inefficient, and lack transparency. Such approaches make it difficult to ensure active participation from all members and do not support proper data management or privacy. With the advancement of digital technologies, there is a growing need for modern solutions that can simplify community governance. A digital platform can provide secure, scalable, and user-friendly features to manage communication, complaints, and decision-making effectively. This project aims to develop such a system by integrating essential functionalities along with advanced features like AI-based assistance, multilingual support, and secure access control to improve overall community interaction and participation.

1.2 PROBLEM STATEMENT

Many communities still rely on outdated manual systems for communication and governance, which lead to delays in information sharing, inefficient decision-making, and poor issue resolution. These methods lack proper organization, transparency, and accessibility, making it difficult for members to actively participate and stay informed. In addition, the absence of structured platforms for handling complaints, petitions, announcements, and voting results in miscommunication and

reduced engagement. Therefore, there is a need for a secure and flexible digital platform that can streamline community activities, ensure transparency, protect user privacy, and provide an efficient and user-friendly environment for all members.

1.3 OBJECTIVES

1.3.1 Digital Community Management

To provide a platform for managing community activities such as discussions, events, announcements, and decision-making in an organized and transparent manner.

1.3.2 Inclusive Communication

To support multilingual communication and role-based access so that all members, regardless of language or role, can participate effectively.

1.3.3 Automated Issue Handling

To use AI-based features such as complaint categorization and response suggestions (LLM with RAG) to simplify and speed up issue resolution.

1.3.4 Secure Digital Governance

To ensure secure operations through authentication, access control, and safe handling of voting and sensitive data.

1.3.5 Real-Time Alerts and Decision Support

To provide instant notifications and updates so that members and administrators stay informed about activities and decisions.

1.3.6 Scalable Administrative Analytics

To include dashboards and analytics for tracking participation, complaints, and system performance to support better decision-making.

1.4 APPLICATIONS

1. Residential Communities

- Apartment management and resident coordination
- Housing society governance and maintenance tracking
- Transparent decision-making processes

2. Educational Institutions

- Student hostel administration and facility management
- University community engagement and participation
- Academic coordination and governance

3. Organizational Management

- Employee engagement and internal communication
- NGO volunteer coordination and project tracking
- Professional association management

4. Local Governance

- Citizen service platforms and complaint handling
- Public participation and consultation systems
- Civic engagement and transparency tools

5. Special Interest Groups

- Social and cultural organization management
- Religious community coordination and planning
- Co-working space administration and member services

CHAPTER 2

LITERATURE SURVEY

2.1 INTRODUCTION

Collaborative governance and shared decision-making have become central themes in empowering communities globally. Digital platforms like AgoraHub have emerged as transformative tools that facilitate inclusive participation, transparent governance, and collective action. The literature on community governance platforms emphasizes the importance of scalable, secure, and user-friendly systems that enable diverse stakeholder engagement through role-based access, automated moderation, and real-time communication.

Recent studies highlight how such platforms leverage emerging technologies, including blockchain, AI, and cloud computing, to enhance transparency, trust, and accountability in governance processes. Despite technological advances, challenges remain in ensuring inclusivity, preventing manipulation, and adapting governance frameworks to diverse community needs. This literature survey explores these themes and reviews state-of-the-art approaches, tools, and frameworks contributing to the design and implementation of platforms like AgoraHub, aiming to foster community empowerment and effective collective decision-making.

2.2 LITERATURE REVIEW OF RECENT WORKS

In [1], the authors propose a novel approach to addressing common challenges in resolving complaints through a mobile-based application. As a mobile application, it is a pivotal component within modern complaint resolution systems, prioritizing efficiency and user satisfaction.

The primary objectives of the application include expediting the complaint resolution process and fostering user contentment. Its core functionality revolves around facilitating the real-time submission of complaints, ensuring swift acknowledgement and resolution. With an intuitive user interface, the application endeavours to simplify the interaction for all users. Ultimately, its overarching goal is to enhance communication channels between citizens and local authorities, ensuring the inclusivity of voices and fostering a more responsive and transparent urban governance framework.

In [2], the authors present a study on the implementation of an application designed to manage community emergency alerts. This study presents the implementation of “ESPE Security,” an application designed to manage community emergency alerts. A data collection technique was employed to understand frequent incidents in the Luz de América parish in the Santo Domingo de los Tsáchilas province. A sample of 440 residents was gathered through surveys covering an age range of 15 to 60 years, including various demographic groups such as students, traders, and the general population. The collected data revealed that approximately 87.95% of the surveyed rural parish population desires an application to report community incidents, as they have reported incidents such as traffic accidents, thefts, crimes, health issues, and homelessness. Based on these findings, a mobile and web application was developed to notify incidents in real time. The design and development of these applications involved technologies such as MongoDB, React, and Node.js, focusing on providing an intuitive user interface. Tests were conducted with 69 end-users and seven experts, employing quantitative techniques to assess design and usability. The results for design were 90%, and for

usability, it was 66.70%. Therefore, mobile and web applications are helpful for real-time notification of critical incidents and will contribute to the community.

In [3], the authors introduce a machine learning-based pipeline for managing customer complaints on digital platforms. In recent years, digital platform companies have faced increasing challenges in managing customer complaints, driven by widespread consumer adoption. This paper introduces an end-to-end pipeline, named RE-GrievanceAssist, designed specifically for real estate customer complaint management. The pipeline consists of three key components: i) response/no-response ML model using TF-IDF vectorization and XGBoost classifier; ii) user type classifier using fasttext classifier; iii) issue/sub-issue classifier using TF-IDF vectorization and XGBoost classifier. Finally, it has been deployed as a batch job in Databricks, resulting in a remarkable 40% reduction in overall manual effort with monthly cost reduction of Rs 1,50,000 since August 2023.

In [4], the authors present an AI-supported extension to an online participation platform to improve the quality of discussions and participant interaction. Online spaces allow people to discuss important issues and make joint decisions, regardless of their location or time zone. However, without proper support and thoughtful design, these discussions often lack structure and politeness during the exchanges of opinions. Artificial intelligence (AI) represents an opportunity to support both participants and organizers of large-scale online participation processes. In this paper, we present an extension of adhocracy+, a large-scale open source participation platform, that provides two additional debate

modules that are supported by AI to enhance the discussion quality and participant interaction.

In [5], the authors explore the concept of smart city governance and the role of digital platforms in improving urban management and community engagement. Smart city governance represents a transformative approach to urban management, leveraging digital platforms and advanced technologies to enhance inclusivity, transparency, and efficiency. This paper explores the foundational principles of smart city governance and the critical role of digital platforms in fostering inclusive community engagement. It examines how IoT, AI, and big data enable real-time decision-making, improving resource allocation and responsiveness to urban challenges. Key findings highlight the benefits of integrating technology into governance frameworks while addressing challenges such as ethical concerns, data privacy, and digital accessibility. The paper concludes with policy recommendations for enhancing governance practices, emphasizing the importance of inclusivity, ethical standards, and public-private collaboration. These insights offer a roadmap for developing sustainable, citizen-centered smart cities capable of adapting to future demands.

In [6], the authors present an application designed to support inclusive decision-making processes in cultural heritage management. This paper presents iCommunity, an application that will be aimed at finding a universal method for inclusive societies in decision-making processes for cultural heritage management. iCommunity is being designed as a mobile application facilitating public participation in cultural heritage management. The main idea is to give all people, who are affected by a decision, the opportunity to be involved in the decision-making process.

Any cultural heritage institution will be able to publish its activities on the application so as to evaluate users' feedback before the implementation of the activity itself. Based on the user's location, the application displays ongoing activities to the user himself. As a part of a participatory approach, users share each other's experience or data related to the posted activity. They also can show their agreement or disagreement to the activities by voting. The comments, votes and user's activities will be analysed real time for helping authorities to consider the consequence of their activities for more transparent and effective decision-making.

2.3 SUMMARY OF LITERATURE SURVEY

Table 2.3 Literature Survey

S.No	Paper Title	Authors	Journal & Year	Techniques Used
1	CitizenConnect: Real-Time Grievance App	Mitesh Shinde, Shubhangi Teli, Mr. Nilesh V. Kamble	IJIRSET, 2024	Secure authentication, real-time data handling, image uploads, dashboards
2	ESPE Security: Mobile and Web Emergency Alerts	Veronica Martinez Cepeda, Maria Parraga	iJIM, 2024	Incident categorization, geolocation-based alerts, real-time reporting
3	RE-GrievanceAssist: ML-powered Complaint Management	Venkatesh C., Harshit Oberoi, Anurag Pandey, Anil Goyal, Nikhil Sikka	arXiv, 2024	Machine learning, NLP for complaint classification and routing
4	Supporting Online	Maike Behrendt, Stefan Wagner,	arXiv preprint,	BERT-based stance detection, AQUA

	Discussions with AI in adhocracy+	Stefan Harmeling	2024	scoring, synthetic data generation (Mistral-7B)
5	Smart City Governance Framework	James Paul Onoja, Olakunle Abayomi	GSC Advanced Research Reviews, 2023	Tamper-proof voting, NLP for complaints, participatory budgeting, federated microservices
6	iCommunity: Public Participation in Museums	A. Nasrolahi, V. Messina, C. Gena	ACM AVIC2CH , 2020	Voting/commenting system, demographic dashboards, gamification

2.4 INFERENCES FROM LITERATURE REVIEW

- Role-Based Access Control: Essential for managing varied participation levels and responsibilities within communities, balancing openness with governance.
- Encrypted Communication: Protects user privacy while enabling secure moderation and governance, crucial in privacy-sensitive communities.
- AI and Automation: Machine learning and natural language processing streamline complaint handling, moderation, and sentiment analysis, reducing human bias and speeding up processes.
- Blockchain & Cloud Integration: Ensures transparency, immutability, and trust in community decisions such as voting and budgeting

through decentralized systems.

- **Customizable Governance:** Platforms must support diverse community types through adaptable governance workflows, multilingual support, and scalable administrative tools.
- **Real-Time Interaction:** Live notifications, discussion tools, and analytics dashboards maintain member engagement and promote accountability.
- **Inclusivity Challenges:** Bridging digital literacy and access gaps remains critical to ensure equitable participation and to prevent exclusion.
- **Data-Driven Insights:** Continuous monitoring allows administrators to improve governance mechanisms based on participation metrics and community feedback

2.5 FINAL OBJECTIVES OF THE PROPOSED SYSTEM

1. **Digitize and Streamline Participation:** Replace manual community processes with an integrated digital platform that simplifies engagement and governance.
2. **Inclusive Communication:** Support multilingual content and role-based access to serve diverse community members effectively.
3. **AI-Powered Moderation:** Employ AI tools for complaint categorization, sentiment analysis, and fair content moderation.

CHAPTER 3

SYSTEM ARCHITECTURE

3.1 ARCHITECTURE OVERVIEW

The proposed Community Governance System is designed to provide a digital platform for managing and improving communication within communities such as hostels, apartments, and clubs. The architecture follows a client–server model, where a React Native mobile application acts as the frontend, and a Node.js + Express.js backend serves as the API layer, connected to a PostgreSQL database. The system enables secure user registration, space creation by the head, member management, complaint submission, and announcement posting. All data operations are handled through RESTful APIs, ensuring reliability, scalability, and ease of maintenance.

3.2 ARCHITECTURE DIAGRAM

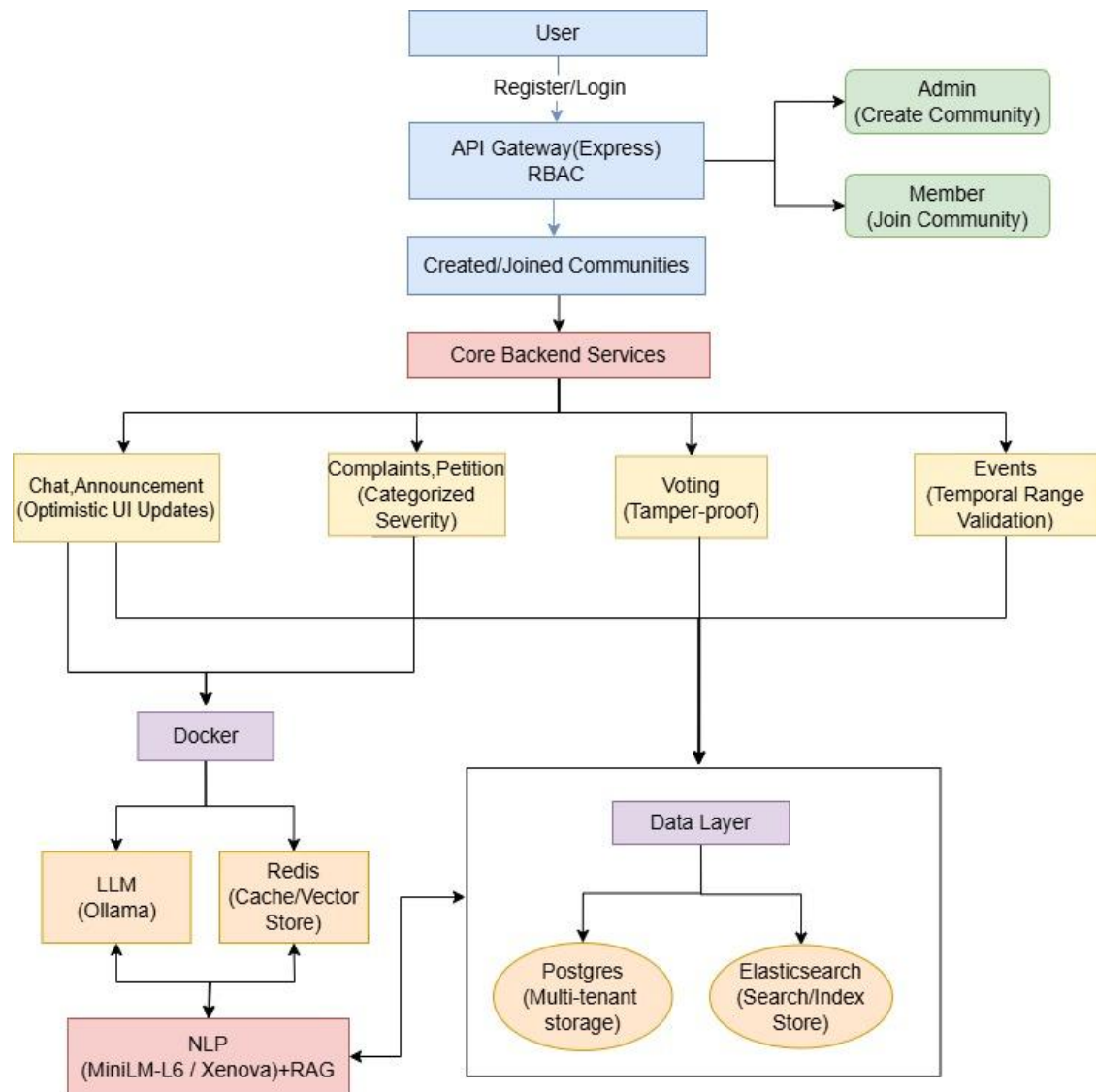


Fig 3.2 System Architecture

3.3 ARCHITECTURE DESCRIPTION

The AgoraHub system follows a modular and scalable architecture that integrates secure authentication, real-time communication, intelligent decision-making, and efficient data management. The architecture is designed to ensure transparency, performance, and reliability in community governance.

3.3.1 User

The user is the primary entity interacting with the system. Users can be categorized as Admin (Head) or Member based on their roles. They access the platform through a mobile interface developed using React Native.

Users perform various operations such as registration, login, joining communities, raising complaints, participating in voting, and engaging in real-time communication. The lightweight user interface ensures smooth interaction and reduces dependency on traditional manual systems, thereby improving usability and efficiency.

3.3.2 Authentication

The system implements a secure authentication mechanism using JSON Web Tokens (JWT) and bcrypt hashing. During registration, user passwords are securely hashed before storage. During login, credentials are verified and a JWT token is generated.

This token is used for all subsequent API requests, ensuring secure and stateless communication. The use of JWT eliminates the need for server-side session storage, improving scalability and reducing overhead.

Mathematically, password security is ensured using hashing:

$$\text{Hash} = \text{bcrypt}(\text{password} + \text{salt})$$

JWT consists of three components:

$$\text{JWT} = \text{Header} + \text{Payload} + \text{Signature}$$

3.3.3 API Gateway

The API Gateway acts as the central request handler using Node.js and Express. It validates incoming requests by verifying JWT tokens and enforcing Role-Based Access Control (RBAC). Each request is checked for authentication and authorization before being forwarded to the appropriate backend service. This centralized approach reduces redundancy and ensures secure and efficient request processing.

3.3.4 Role Management

Role management controls access permissions within the system. The main roles include Admin and Member. Admin users have privileges such as creating communities and managing activities, while Members have limited access. The RBAC mechanism ensures that only authorized users can perform specific actions, enhancing system security and preventing misuse.

3.3.5 Community Management

This module manages the lifecycle of community spaces. Admin users can create communities, and members can join using invite codes or approval mechanisms. The system follows a multi-tenant architecture, where data for each community is isolated using unique identifiers. This ensures scalability and maintains data integrity across multiple communities.

3.3.6 Core Backend Services

The core backend services form the main processing layer of the system. It includes modules such as chat, complaints, anonymous messaging, voting, events, and notifications.

All requests from the API Gateway are processed here and forwarded to the respective modules. The modular design allows independent scaling and efficient load management.

3.3.6.1 Chat and Announcement Module

This module provides real-time communication using Socket.io. Users can send messages instantly, and admins can broadcast announcements to all members. Messages are stored in the database for future reference. The use of real-time communication ensures low latency and improves user engagement.

3.3.6.2 Complaint and Petition Module

The Complaint and Petition Module is a cornerstone of the AgoraHub platform, designed to empower community members by giving them structured, actionable channels to voice concerns and propose community changes. It acts as a bridge between the users and the administration, transforming passive feedback into trackable, resolvable actions.

Key Features & Workflow:

1. Complaint Management System:

- ***Structured Reporting:*** Users can submit detailed complaints regarding community issues (e.g., maintenance problems, rule

violations, or general concerns). Submissions can include categorical tags, detailed text descriptions, and multimedia evidence (like photos).

- ***AI-Assisted Routing (Integration):*** Through integration with the platform's Natural Language Processing (NLP) pipeline, incoming complaints are automatically scanned. The system can immediately detect the urgency or sentiment of the complaint to prioritize it for administrative review, ensuring critical issues are not buried in the queue.
- ***Transparent Tracking Lifecycle:*** Once submitted, a complaint enters a tracked lifecycle with clear statuses: Submitted, Under Review, In Progress, and Resolved. Users are kept informed via automated notifications whenever the administrative team updates the status of their submitted complaint.

2. Community Petitions:

- ***Democratic Proposals:*** Unlike private complaints, petitions are public proposals designed to rally community support for specific changes, new rules, or large-scale initiatives.
- ***Threshold & Escalation:*** Users can digitally "sign" or support active petitions. Administrators can configure specific participation thresholds (e.g., 100 signatures or 20% of the community). Once a petition crosses this customized threshold, it is automatically escalated to the leadership team or officially transitioned into the Voting Module for a binding community decision.

3. Administrative Dashboard: The module provides community leaders with a centralized dashboard. Instead of dealing with unorganized emails or physical complaint boxes, admins can view analytics on the most

frequent types of complaints, track petition momentum, and manage community relations through an efficient, data-driven interface.

3.3.6.3 Voting Module

The voting module is designed to facilitate democratic, secure, and fair decision-making processes across the community. Whether used for leadership elections, policy changes, or casual polls, it guarantees an uncompromising level of integrity and transparency.

Key Features & Workflow:

- ***Enforced Tamper- Proof Mechanism:*** The system intrinsically ties voting rights to verified user accounts. Through secure tokenization, it ensures that each authenticated user is strictly allowed to cast only one ballot per election, mathematically eliminating the possibility of duplicate voting.
- ***Blind Tallying & Trend Protection:*** To prevent the "bandwagon effect" and ensure impartial decisions, real-time vote counts are hidden during the active voting period. Votes are securely sealed and are only decrypted and counted by the system after the officially scheduled voting process has concluded.
- ***Election Lifecycle Management:*** Administrators can define the parameters of a poll, including descriptions, choices, and strict start/end timestamps.
- ***Auditability & Transparency:*** Once the election window closes, the results are automatically tallied and published to the community. While individual votes remain strictly anonymous, the aggregated data is presented transparently, guaranteeing the community that the outcome is fair and mathematically sound.

3.3.6.4 Events Module

The events module acts as the central hub for community scheduling and activities. It replaces fragmented communication by offering a structured, centralized system where administrators can effectively create, manage, and distribute information regarding meetings, social gatherings, and official activities.

Key Features & Workflow:

- ***Comprehensive Event Creation:*** Administrators can generate rich event profiles that include essential details such as the event title, date, time, physical location (or virtual meeting links), comprehensive agendas, and participant capacity limits.
- ***User Participation & RSVP:*** Members have access to a unified calendar or list interface where they can browse upcoming events, view specific details, and explicitly indicate their participation status (RSVP). This allows organizers to accurately estimate headcount and allocate resources.
- ***Automated Communication:*** The module bridges communication gaps by supporting automated reminders and notifications. Users can be alerted of upcoming events they have registered for, or notified instantly if there are sudden changes to the schedule or venue.

3.3.7 Docker

To ensure maximum structural integrity and deployment reliability, the AgoraHub platform utilizes Docker as its primary containerization technology. By encapsulating the application code alongside its runtime, system tools, libraries, and settings, Docker guarantees environmental consistency across development, testing, and production stages.

Key Features & Workflow:

- ***Modular Isolation:*** The platform's architecture is deeply decoupled. Critical components—specifically the Node.js backend server, the Redis caching service, and the heavy-duty LLM inference engines—run in entirely separate, logically isolated containers. This prevents dependency conflicts and ensures that the failure of one service does not indiscriminately crash the entire system.
- ***Structured Deployment Pipeline:*** The deployment lifecycle follows a strict, automated progression:
 1. Source Code Integration
 2. Dockerfile Execution & Dependency Installation
 3. Immutable Docker Image Compilation
 4. Container Initialization & Health Checking
 5. Service Integration (binding the Database, Redis, and LLM limits)
 6. Live Application Deployment
- ***Internal Network Channels:*** Containers do not communicate over raw public network interfaces. Instead, they operate on defined, virtual bridged networks (e.g., Backend Container ↔ Redis Container and Backend Container ↔ LLM Container). This creates a closed-loop security environment, allowing rapid, secure data exchange while shielding internal services from external exposure.
- ***Dynamic Scalability:*** The containerized nature of the project allows for horizontal scaling. During peak traffic (e.g., a highly contested voting period or viral community event), a single Node.js backend

service can be effortlessly replicated into multiple parallel containers to efficiently handle massive concurrent user requests.

3.3.7.1 Large Language Model

The platform departs from traditional reliance on commercial cloud AI by integrating a powerful, localized Large Language Model (LLM) powered by Ollama. This strategic architectural choice prioritizes absolute user data privacy while delivering highly intelligent, conversational capabilities natively within the AgoraHub infrastructure.

Key Features & Workflow: Retrieval-Augmented Generation (RAG): Standard LLMs can hallucinate or lack domain-specific knowledge. AgoraHub relies on a robust RAG architecture. Before generating a response, the system queries the internal database for relevant, up-to-date context regarding specific community data, rules, or recent petitions.

- ***Execution Pipeline:*** The secure flow of contextual AI processing operates as follows:
- ***User Query Submitted:*** (e.g., asking about a specific local complaint or petition).
- ***Context Retrieval:*** The system fetches highly relevant snippets from the local database.
- ***Prompt Synthesis:*** User input and the fetched contextual data are merged.
- ***Local LLM Processing:*** The model synthesizes a fully informed response based strictly on the provided facts.
- ***Output to User:*** Delivery of linguistically accurate, highly contextualized information.

- ***Zero-Latency Local Inference:*** Because all AI inferencing is processed entirely on the host server hardware—rather than sending massive payloads over the internet to remote API providers—the system drastically reduces network latency overhead. This guarantees rapid, sub-second response times while ensuring zero user data ever leaves the local environment.

3.3.7.2 Redis

To mitigate processing bottlenecks and safeguard database bandwidth, the system leverages Redis—an advanced, exceptionally fast in-memory data store. Redis acts as the primary buffer between the user-facing web server and the heavier underlying databases and machine learning models.

Key Features & Workflow: Intelligent Query Caching: Redis is tasked with memorizing the outputs of highly complex or repetitive operations (such as deep NLP results, aggregated user session data, and complex database lookups).

The Cache Resolution Cycle:

- ***Incoming Request:*** System identifies the needed data.
- ***Cache Interrogation:*** The system asks Redis if it holds a recent copy of this data.
- ***If Cache Hit:*** The data is returned in milliseconds, safely bypassing the main database entirely.
- ***If Cache Miss:*** The system runs the heavy query/process, returns the output to the user, and simultaneously writes the result back into Redis for the next user.

Security via Rate Limiting: Beyond simple caching, Redis functions as a critical security governor. By tracking incoming request IPs within rolling time windows, Redis natively handles rate limiting. If a user or bot submits sudden, overwhelming amounts of requests, Redis intercept and throttles them, preventing Distributed Denial of Service (DDoS) scenarios and ensuring fair resource distribution.

3.3.7.3 NLP and RAG System

At the core of AgoraHub's proactive moderation and user assistance lies a sophisticated Natural Language Processing (NLP) framework. Utilizing advanced transformer-based machine learning models, the system seamlessly analyzes unstructured text to ensure a safe and supportive community environment.

Key Features & Workflow:

- ***Concurrent Text Analysis:*** When a user submits content (posts, comments, complaints), the text is not simply stored; it is deeply analyzed. The pipeline executes multiple tasks sequentially and concurrently:
- ***Language Detection:*** Establishing the foundational dialect of the text to route it to the correct processing model.
- ***Parallel Processing Engine:*** The text is routed simultaneously to a Sentiment Classifier (to gauge community mood: positive, negative, neutral) and a Toxicity Detector (to flag hate speech, severe profanity, or harassment).
- ***Result Synthesis:*** The outputs are combined to evaluate the payload.

- ***Output Generation:*** The content is either approved, flagged for admin review, or utilized directly to improve the context fed into the RAG pipeline.
- ***Performance Optimization (NLP Caching):*** Running deep-learning transformer models is computationally expensive. Therefore, the outputs of all text scans are deeply integrated with the Redis caching layer, ensuring that repeatedly viewed or analyzed text blocks do not trigger redundant system strain.
- ***Fault-Tolerant Fallback Mechanisms:*** Designed for high availability, the platform features internal safety nets. If an NLP container crashes or fails to respond under heavy load, the system degrades gracefully. It bypasses the analysis layer temporarily, allowing core application operations (like posting emergency messages) to function without interruption until the AI container automatically restarts.

3.3.8 Data Storage Layer

The Data Storage Layer is a critical component of the AgoraHub system, responsible for securely storing, managing, and retrieving all application data. It ensures data

consistency, scalability, and efficient access across all modules such as authentication, chat, complaints, voting, and AI processing. The system follows a hybrid storage architecture by combining PostgreSQL for structured data and MinIO for unstructured data. This separation improves overall system performance and optimizes data handling based on the nature of the data.

3.3.8.1 PostgreSQL

PostgreSQL is a relational database management system used to store structured and transactional data in the form of tables. In the AgoraHub system, it manages all core application data including user details (ID, name, role, authentication information), community data (community ID, members, roles), complaints and petitions (title, description, status), chat messages and announcements, voting records, and event details. The system uses a multi-tenant architecture, where each community's data is logically separated using a unique tenant identifier. This ensures data isolation, improves security, and supports scalability across multiple communities.

PostgreSQL follows a structured execution pipeline. Initially, the SQL query is parsed and analyzed. Then, a query planner determines the most efficient execution strategy. After planning, the execution engine performs the required operations such as data insertion or retrieval. Finally, transaction management ensures that all operations maintain consistency.

The complete workflow can be described as follows:

User Request (e.g., complaint submission)

- API Gateway
- Backend Service
- SQL Query Generation
- PostgreSQL Execution
- Data Stored or Retrieved
- Response Returned to User

To improve query efficiency, PostgreSQL uses indexing techniques such as B-Tree indexing for general queries, hash indexing for equality comparisons, and GIN indexing for advanced operations like vector search. These indexing methods reduce time complexity from $O(n)$ to $O(\log n)$, significantly improving performance.

As a result, PostgreSQL provides:

- Fast query execution
- High data integrity
- Efficient multi-user access
- Optimized relational data processing

3.4 DATA FLOW DIAGRAM

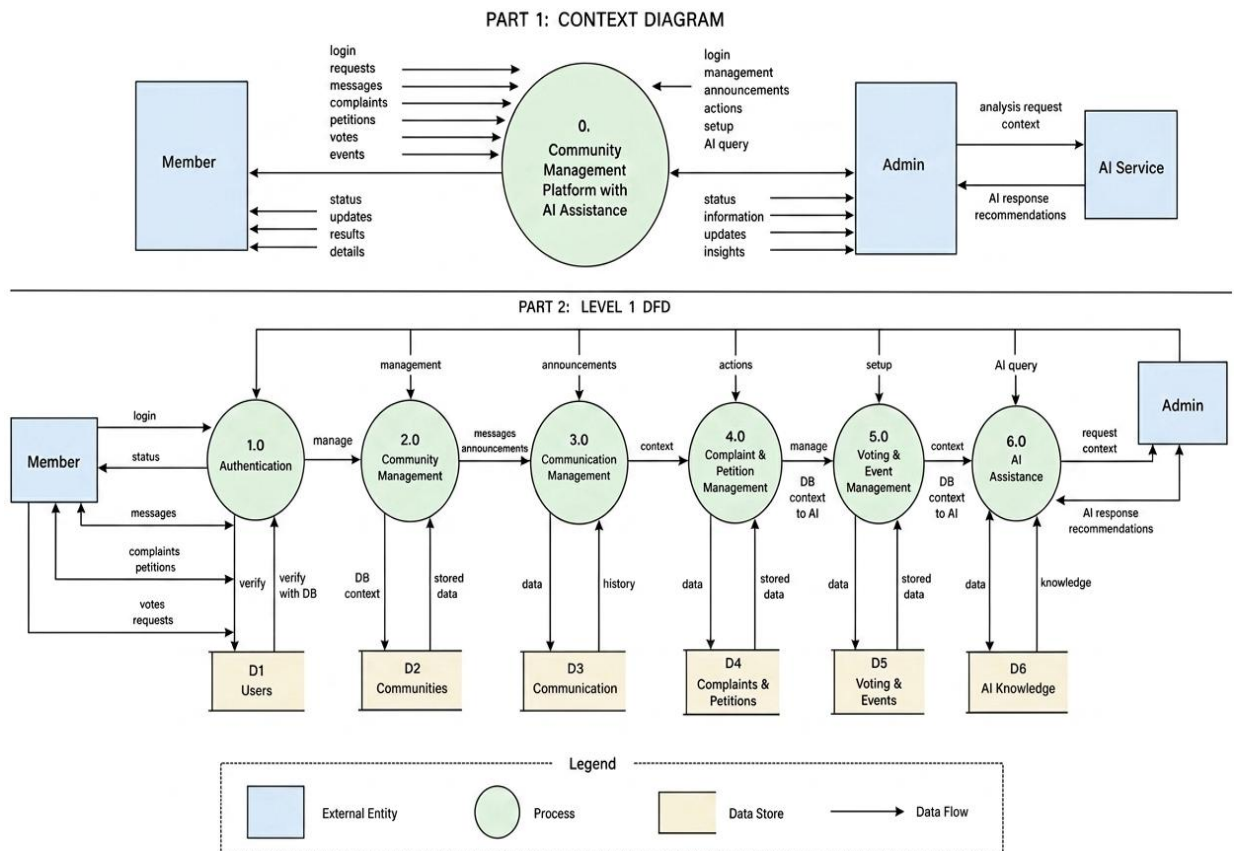


Fig 3.4 Data Flow Diagram

3.5 USECASE DIAGRAM

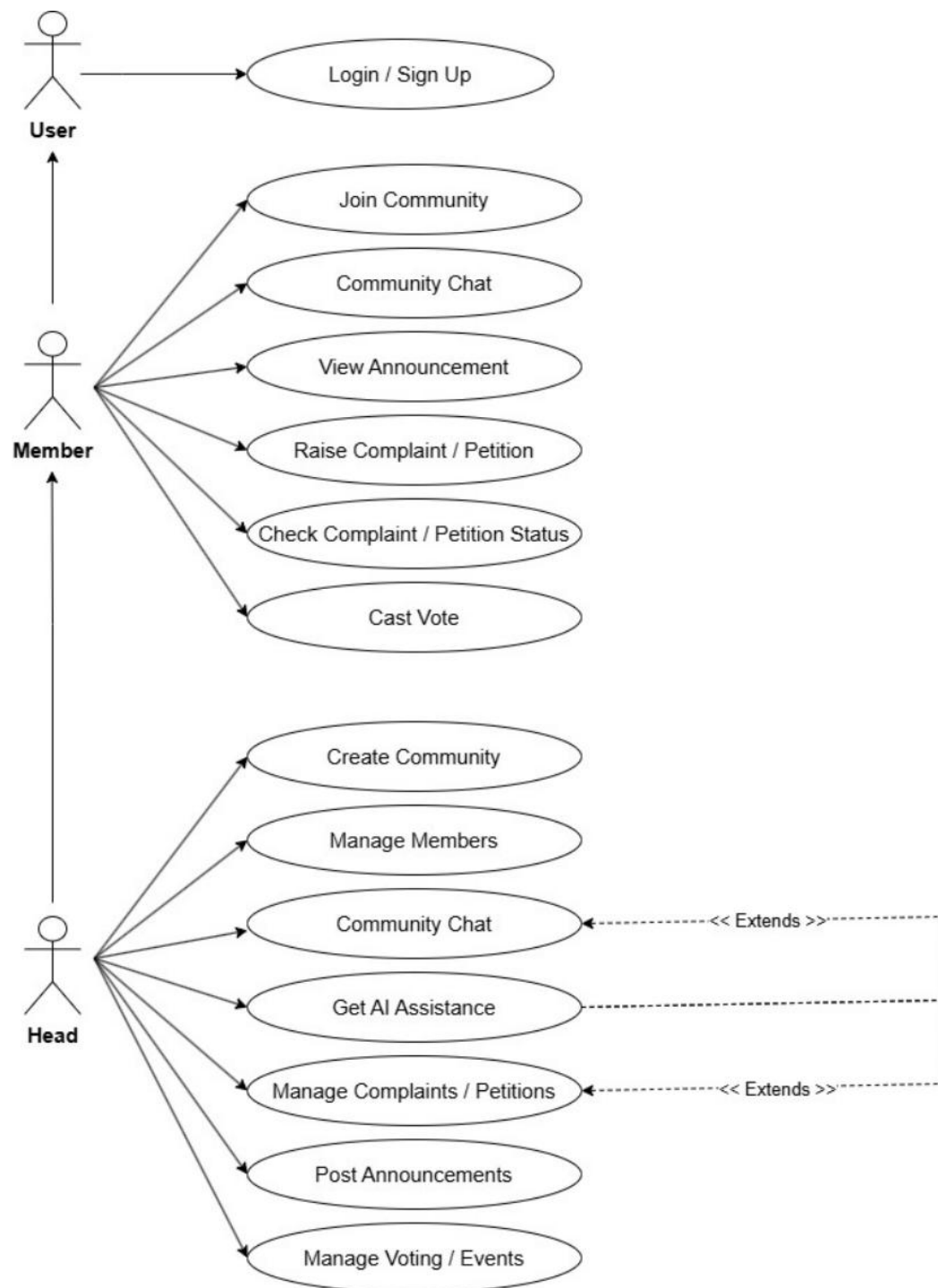


Fig 3.5 Usecase Diagram

CHAPTER 4

SYSTEM REQUIREMENTS

4.1 HARDWARE REQUIREMENTS

- Processor: Intel(R) Core i5 or higher
- RAM: Minimum 8 GB
- Storage: 20 GB free space
- Network Adapter: Required for internet connectivity

4.2 SOFTWARE REQUIREMENTS

- Operating System: Windows / macOS / Linux
- Frontend Framework: React Native
- Backend Framework: Node.js with Express.js
- Database: PostgreSQL
- Version Control: Git / GitHub
- AI Infrastructure: LLM (via Ollama)

CHAPTER 5

SYSTEM DESIGN AND IMPLEMENTATION

5.1 OVERVIEW OF THE SYSTEM DESIGN

The AgoraHub Community Management System is designed to streamline communication and governance within shared spaces such as hostels, apartments, and clubs. It enables users to register, create or join communities, raise complaints, share announcements, and participate in collaborative discussions through an intuitive mobile platform.

The system follows a modular, layered architecture consisting of:

- User Interface Layer
- Application Logic Layer
- Community Management Layer
- Messaging & Announcement Layer
- Data Storage Layer

The application is built using React Native for the mobile frontend, Node.js with Express.js for backend processing and API services, and PostgreSQL for reliable data storage and management. Each module operates independently yet integrates seamlessly to provide secure authentication, structured communication, and transparent community coordination.

5.2 MODULES

5.2.1 User Authentication & Registration Module

This module handles secure user registration and login processes. It uses JWT-based authentication to ensure that only authorized users can access

system features. User credentials are validated properly, and roles such as Head and Member are assigned

during account creation. It also manages user sessions to improve security and provide a smooth user experience.

5.2.2 Community Management Module (Create / Join Space)

This module allows Heads to create new community spaces such as hostels, apartments, or clubs. Members can join these spaces using invite codes or through approval by the Head. It stores important details like community name, type, and creation date. The module ensures that users can participate in one or more communities while maintaining proper access control and structured communication.

5.2.3 Chat & Announcement Module

This module provides a built-in chat system that enables members to communicate within the community. It supports real-time messaging for effective coordination. Heads or Admins can post announcements that are visible to all members. All messages and announcements are securely stored in the database for future reference, ensuring reliable communication without depending on external platforms.

5.2.4 Complaint & Petition Module

This module enables members to raise complaints or petitions related to community issues. Each entry includes details such as title, description, status, and submission date. Admins or Heads can review, update, approve, or resolve these issues through a structured interface. The module maintains complete records to ensure transparency, accountability, and efficient tracking of all requests.

5.2.5 Events Module

This module allows Heads or Admins to create and manage events such as meetings, celebrations, or maintenance activities. It stores event details including title, date, time, and description. Members can view upcoming and past events, helping improve participation and coordination within the community.

5.2.6 AI Module

The AI module is an independent Node.js service that automatically analyzes incoming user text to detect sentiment and identify toxic language using algorithmic filters. It uses a local Large Language Model (Ollama) paired with a PostgreSQL-driven Retrieval-Augmented Generation (RAG) system to fetch relevant community documents and generate context-aware moderation responses. To maintain high performance, the module extensively caches outputs in Redis while logging all final analytical data into the database for administrative review.

5.2.7 Voting

The Polling Module is a real-time, highly interactive feature that allows communities to engage through visually appealing and fully synchronized polls. Users can seamlessly cast or update their votes while viewing dynamic, animated result charts. Administrators are equipped with a powerful creation interface offering extensive control over poll duration, privacy rules, voting restrictions, and result visibility.

CHAPTER 6

RESULTS AND DISCUSSIONS

6.1 HOME PAGE

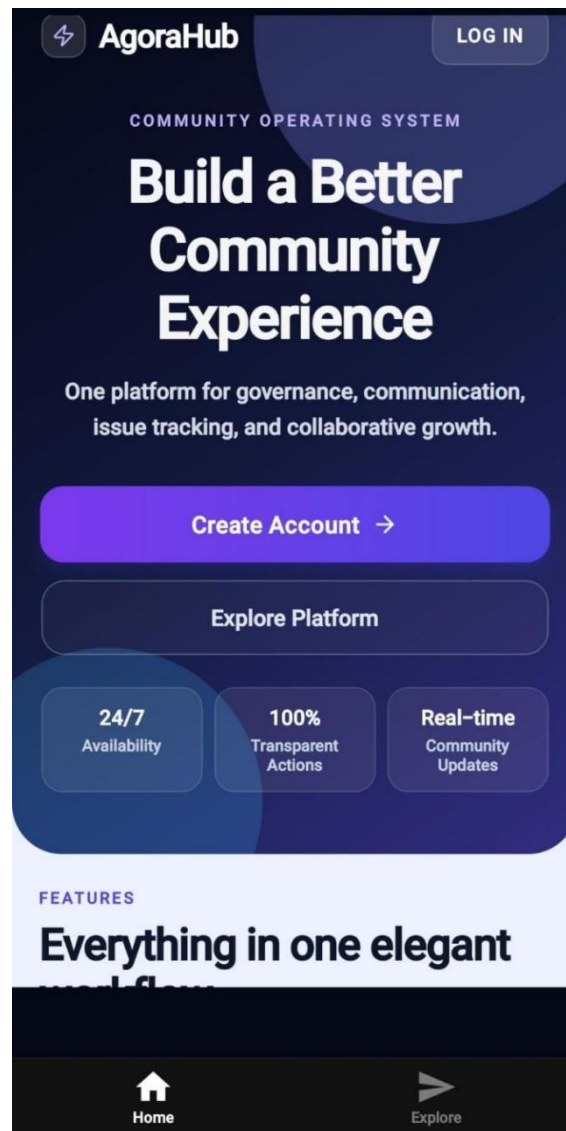


Fig 6.1 Home Page

This page presents AgoraHub as an all-in-one "Community Operating System" designed for governance, communication, and issue tracking. It features a modern interface that prioritizes transparency and real-time updates to help users build a collaborative community experience.

6.2 MANAGE MEMBERS

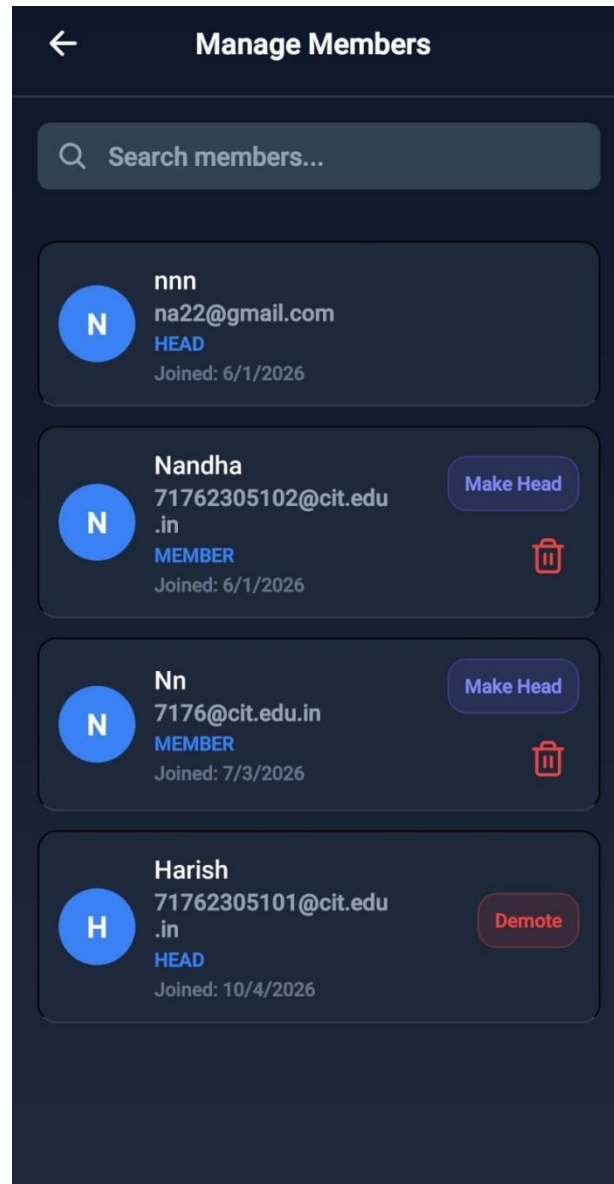


Fig 6.2 Manage Members

The page displays a "Manage Members" interface that allows administrators to search, promote, or remove community members within the AgoraHub platform. It provides a clear overview of member roles, join dates, and contact information, facilitating efficient user management.

6.3 VOTING

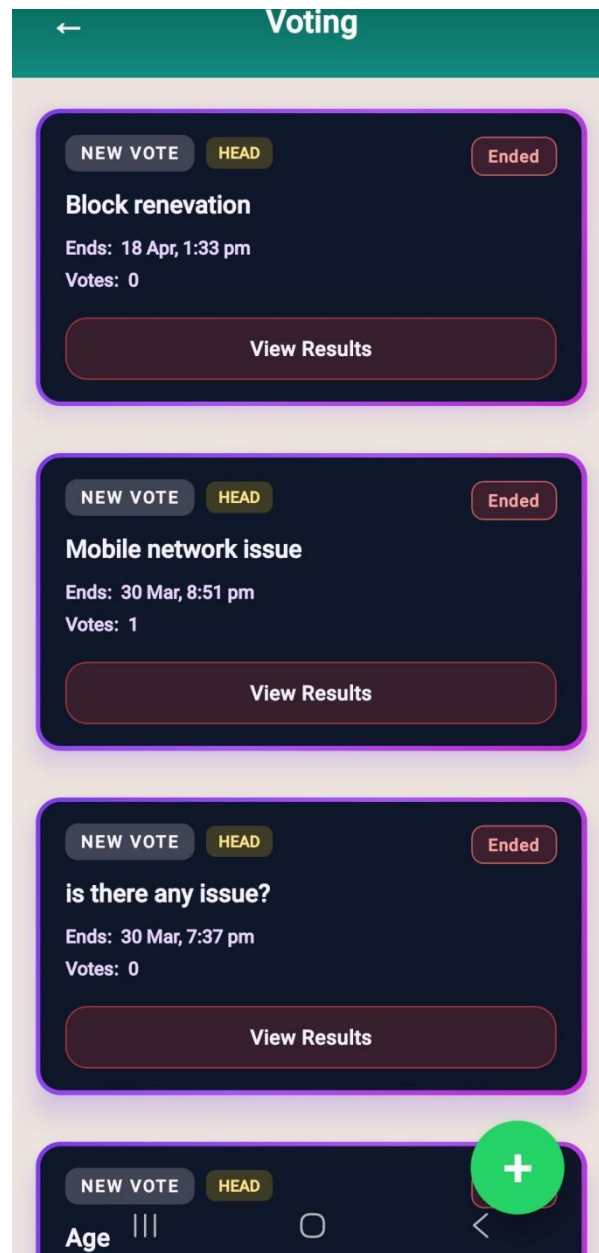


Fig 6.3 Voting

This page displays a "Voting" management screen that lists past community polls, their final vote counts, and expiration details. This interface allows administrators to review feedback on specific topics and initiate new polls to drive collaborative community decisions.

6.4 COMPLAINT AI ASSISTANT

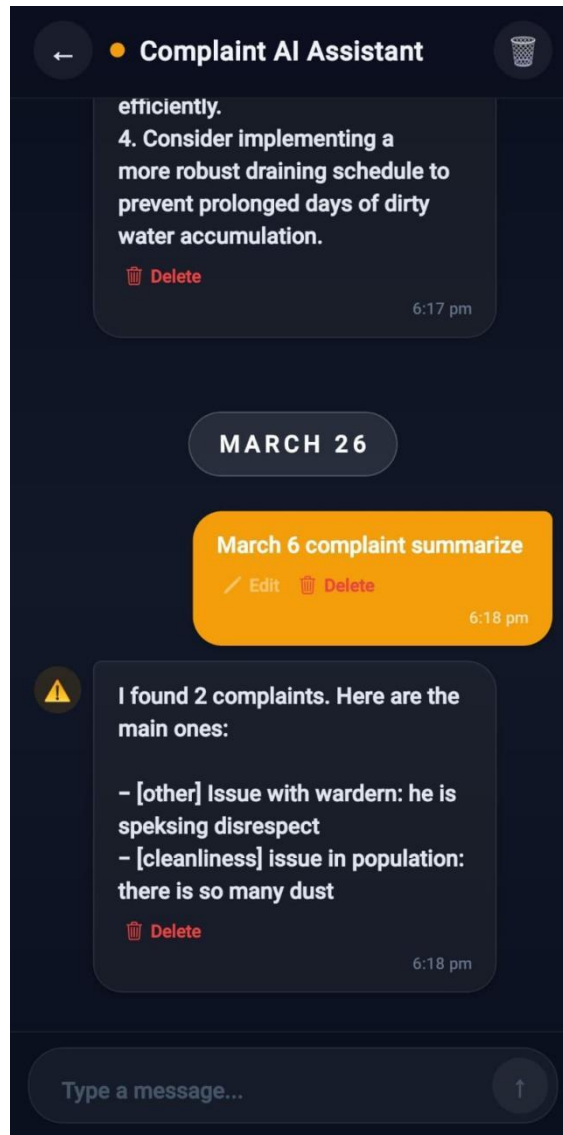


Fig 6.4 Complaint AI Assistant

This page showcases a "Complaint AI Assistant" chat interface that uses artificial intelligence to summarize and categorize user feedback. The tool provides actionable insights, such as maintenance suggestions, and retrieves specific logs regarding personnel behavior and cleanliness to streamline community management.

6.5 EVENTS

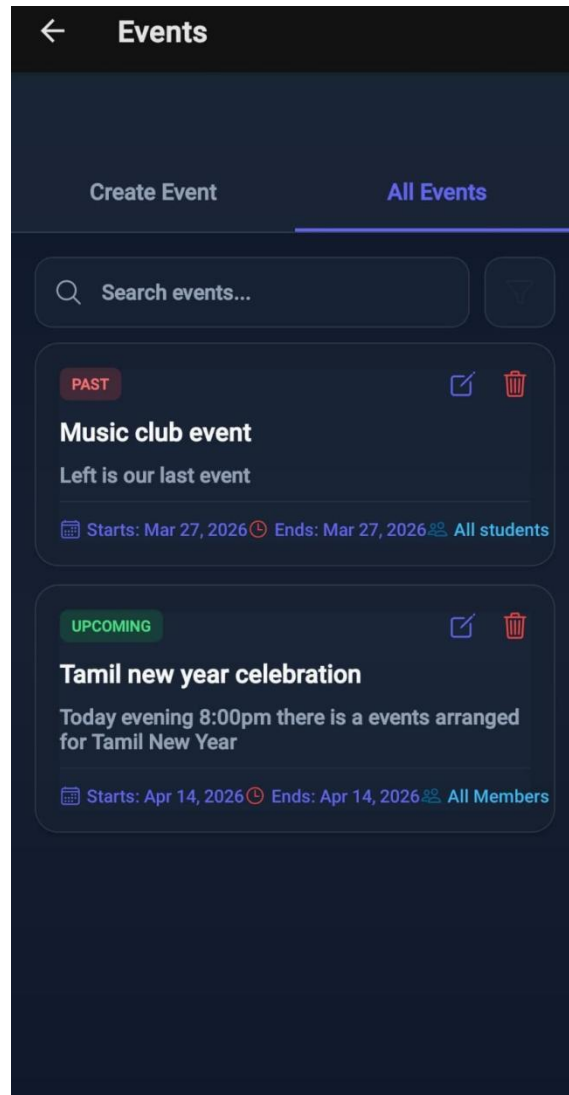


Fig 6.5 Events

This page displays an "Events" management dashboard that allows users to create and browse community gatherings. The interface organizes events by status, such as "PAST" or "UPCOMING," and provides administrative options to edit or delete entries for streamlined event coordination.

6.6 COMMUNITIES

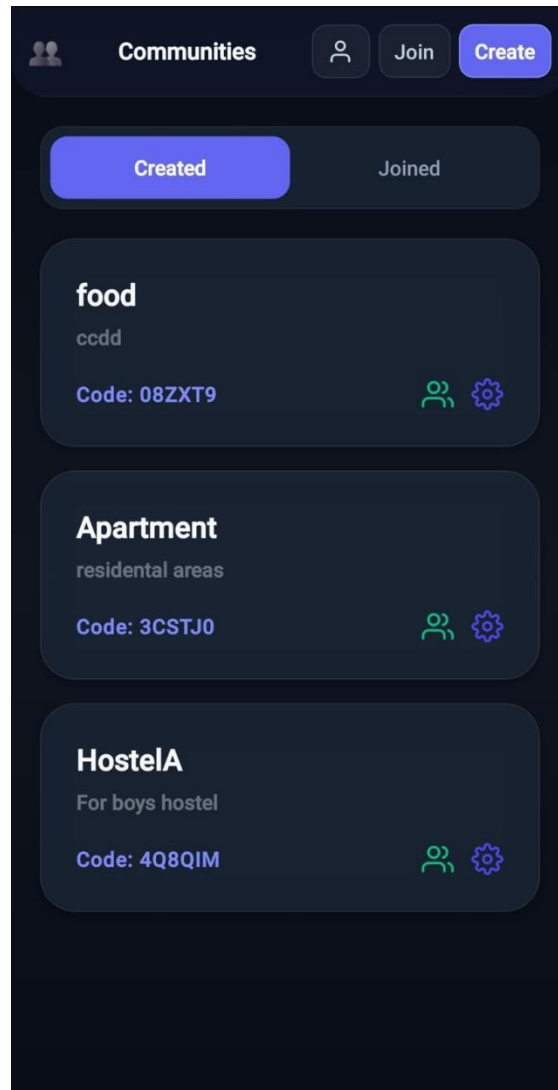
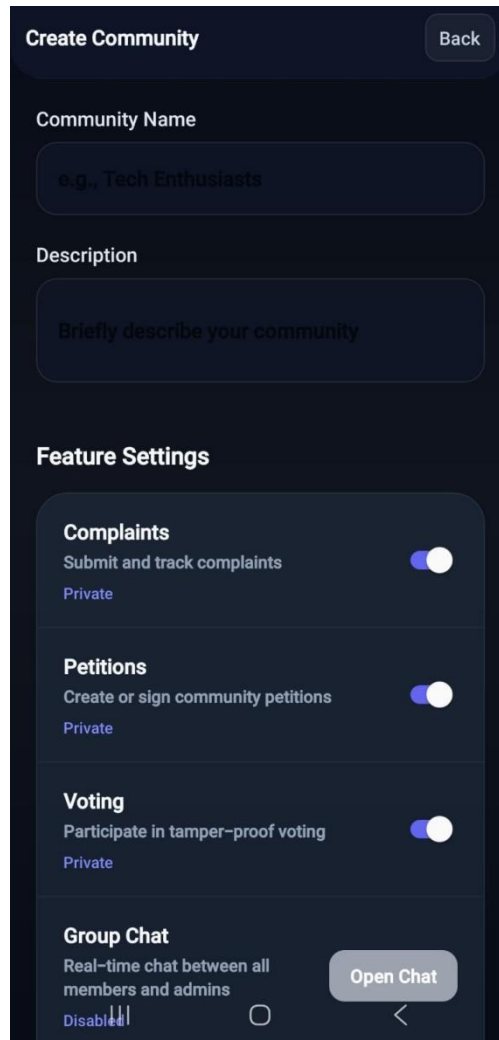


Fig 6.6 Communities

The provided page, displays a "Communities" management screen within the AgoraHub platform. It allows users to toggle between "Created" and "Joined" community lists, providing unique access codes and settings options for groups like "Apartment" and "HostelA".

6.7 CREATE COMMUNITY



The screenshot shows a mobile app interface for creating a new community. At the top, there's a header bar with the title "Create Community" and a "Back" button. Below the header, there are two input fields: "Community Name" with a placeholder "e.g., Tech Enthusiasts" and "Description" with a placeholder "Briefly describe your community". Underneath these is a section titled "Feature Settings" which contains four toggleable features: "Complaints" (Submit and track complaints, Private, toggle on), "Petitions" (Create or sign community petitions, Private, toggle on), "Voting" (Participate in tamper-proof voting, Private, toggle on), and "Group Chat" (Real-time chat between all members and admins, Disabled, toggle on, with an "Open Chat" button). The interface is dark-themed with white text and blue accents for the toggles.

Fig 6.7 Create Community

This page illustrates the "Create Community" interface, where users can define a new group's name and description. It includes "Feature Settings" to enable or disable core functionalities such as private complaints, community petitions, tamper-proof voting, and real-time group chats.

6.8 COMMUNITY APP

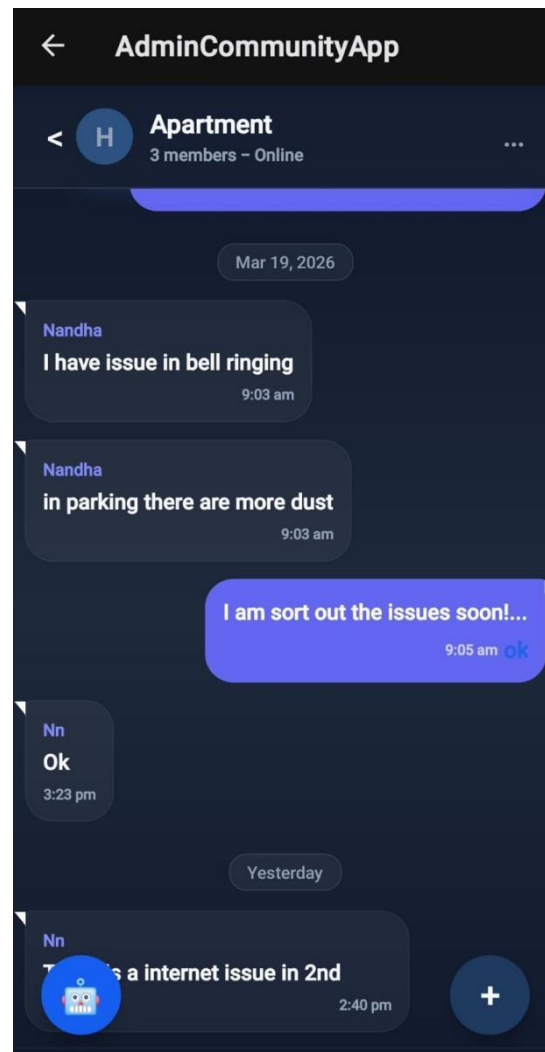


Fig 6.8 Community App

This page, features the "AdminCommunityApp" chat interface for an "Apartment" group, showing real-time communication between members and an administrator. The chat logs document various residential concerns—including doorbell malfunctions, parking lot cleanliness, and internet connectivity—while an admin provides updates on resolution status.

6.9 VOTING ANALYSIS

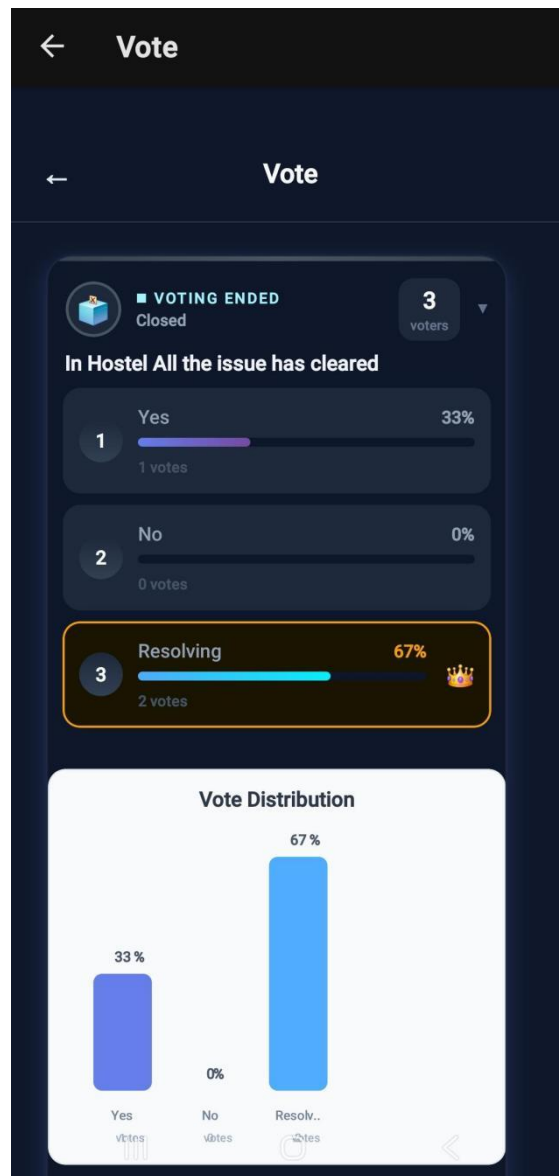


Fig 6.9 Voting Analysis

The page displays a "Voting" results screen that provides a detailed breakdown of community votes for ended polls. It features a "Vote Distribution" bar chart and percentage indicators to visualize survey outcomes, such as an "Age" poll where 100% of participants selected the same option.

6.10 VIEW COMPLAINTS

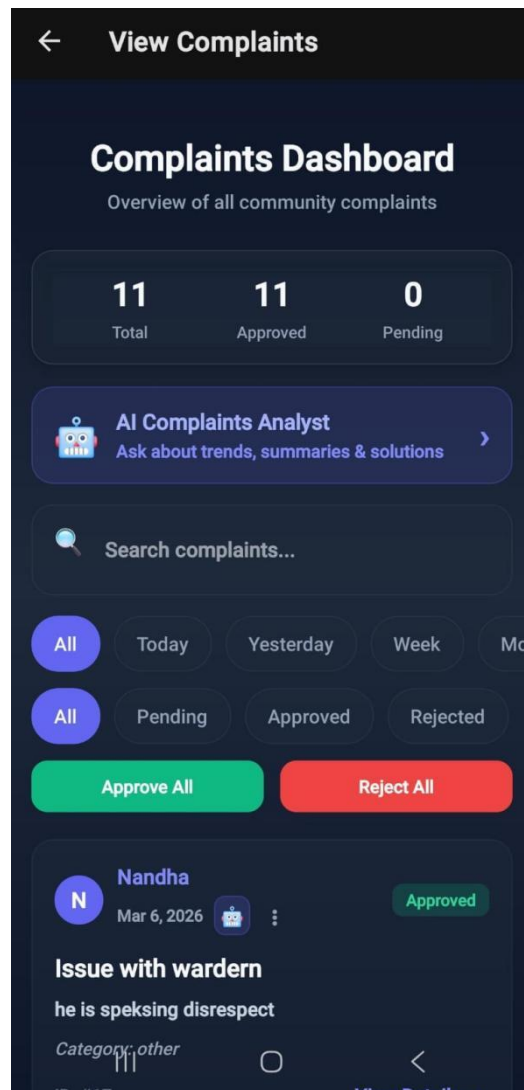


Fig 6.10 View Complaints

This displays a "Complaints Dashboard" for community management, featuring high-level metrics and automated analysis tools. This interface provides an overview of issues, including an "AI Complaints Analyst" to identify trends and a filtered list for managing individual reports.

6.11 AI CHAT

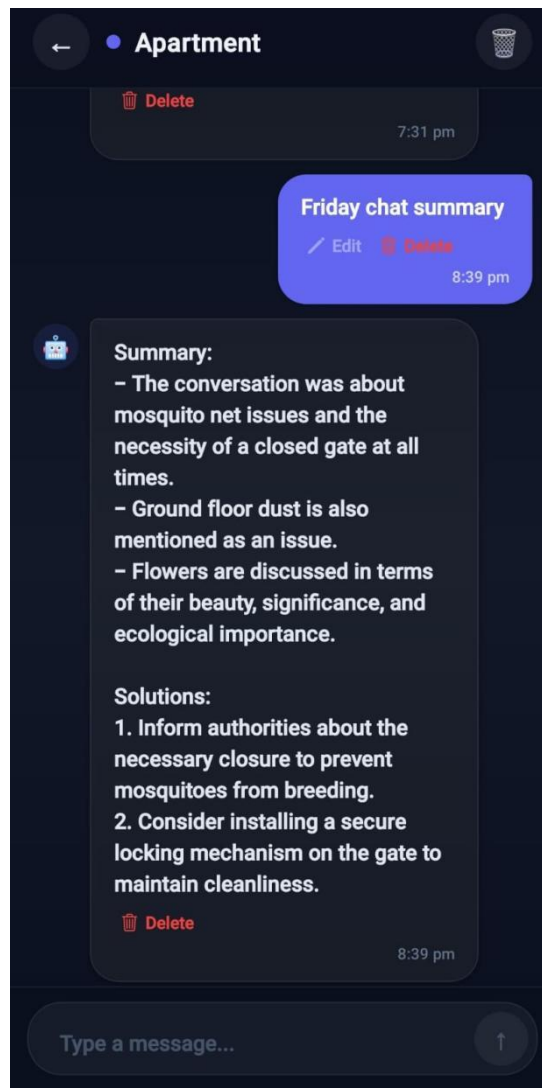


Fig 6.11 AI Chat

This page features an automated chat summary within the "Apartment" community group that condenses resident discussions into key issues and actionable solutions. This specific summary highlights concerns regarding mosquito net maintenance, gate security, and ground floor dust, while proposing measures like implementing secure locking mechanisms to improve local cleanliness and safety.

6.12 CREATE VOTING - BASIC

Close button (X)

Create Voting

Basic Advanced

Voting Question

What would you like to ask the community?

0/200

Context (Optional)

Add more details to help members decide...

0/500

Launch Voting

0 steps remaining

Fig 6.12 Create Voting - Basic

The page shows the "Create Voting" setup screen for community administrators. Users can enter a primary voting question and optional context to help members make informed choices. Once the details are filled in, the poll can be initiated by clicking the "Launch Voting" button.

6.13 CREATE VOTING - ADVANCED

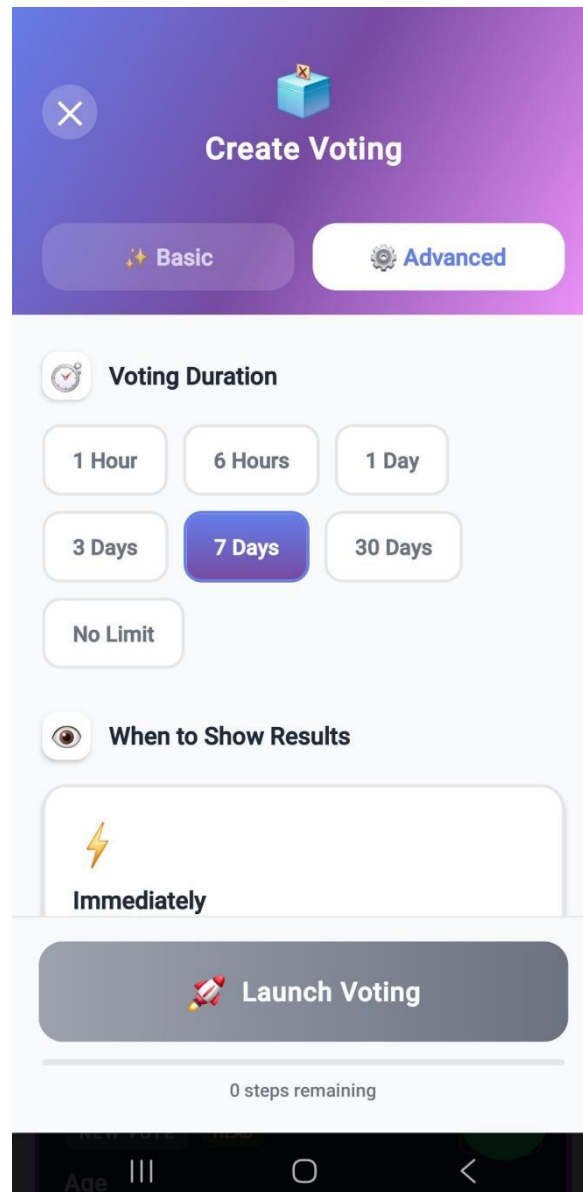


Fig 6.13 Create Voting - Advanced

This page displays the "Advanced" settings for the "Create Voting" interface on the AgoraHub platform. This screen allows users to select a voting duration ranging from "1 Hour" to "30 Days" or "No Limit," while also configuring results to be shown "Immediately". Once these parameters are set, the "Launch Voting" button is used to finalize and start the poll.

CHAPTER 7

CONCLUSIONS AND FUTURE WORKS

7.1 CONCLUSION

The AgoraHub Community Management System provides a unified and practical solution for improving communication and governance within communities such as hostels, apartments, and clubs. Traditional methods like notice boards and paper-based complaints are often slow, unorganized, and lack transparency. AgoraHub addresses these issues by introducing a structured, role-based digital platform that supports complaint management, announcements, community creation, and member interaction in a secure and organized manner.

The system enables users to register, join communities, and actively participate in discussions and issue reporting, while administrators can efficiently manage activities, monitor engagement, and share important updates through an intuitive interface. Built using React Native, Node.js with Express.js, and PostgreSQL, the platform ensures scalability, flexibility, and reliable performance. Additionally, the integration of intelligent features such as LLM with RAG in chat, complaints, and petitions enhances user support by providing relevant suggestions and improving response efficiency.

Overall, AgoraHub offers an effective approach to modern community governance by combining digital accessibility, transparency, and intelligent assistance. It promotes better participation, faster issue resolution, and more organized administration, making it suitable for real-world community environments.

7.2 FUTURE WORK

While the current system meets its main objectives, several enhancements can be introduced in future versions to improve functionality, user experience, and safety. These additions will make AgoraHub more intelligent, interactive, and community-focused.

i) Cloud Storage

Cloud storage can be integrated into the system to improve scalability and data accessibility. It allows data to be stored securely on remote servers instead of local systems. This ensures better data backup, recovery, and availability from anywhere. It also supports handling large volumes of data efficiently without affecting system performance.

ii) Multilanguage Support

Multilanguage support enables the system to be accessible to users from different language backgrounds. It allows users to interact with the application in their preferred language. This improves user experience and increases system usability across diverse communities..

iii) SOS Alert and Notification System

An SOS alert system can be added to send emergency notifications to all members or authorities during critical situations such as medical emergencies, fire, or safety threats. Real-time push notifications can help ensure quick response and enhance community safety.

In summary, these future enhancements will transform AgoraHub into a more advanced, responsive, and intelligent community management platform — fostering safety, transparency, and smarter collaboration.

REFERENCES

1. M. Shinde, S. Teli, and N. V. Kamble, “CitizenConnect: Real-Time Grievance Management App,” *International Journal of Innovative Research in Science, Engineering and Technology (IJIRSET)*, 2024,pp4-5.
2. V. M. Cepeda and M. Parraga, “ESPE Security: Mobile and Web Emergency Alerts,” *International Journal of Interactive Mobile Technologies (iJIM)*, 2024,pp5-6.
3. V. C., H. Oberoi, A. Pandey, A. Goyal, and N. Sikka, “RE-Grievance Assist: ML-powered Complaint Management,” *arXiv preprint*, 2024,pp6.
4. M. Behrendt, S. Wagner, and S. Harmeling, “Supporting Online Discussions with AI in adhocracy+,” *arXiv preprint*, 2024,pp6-7.
5. J. P. Onoja and O. A. Abayomi, “Smart City Governance Framework,” *GSC Advanced Research and Reviews*, 2023,pp7.
6. A. Nasrolahi, V. Messina, and C. Gena, “iCommunity: Public Participation in Museums,” in *Proc. ACM AVI Conference (AVIC2CH)*, 2020,pp7-8.